



Testing

Proceso destructivo, el objetivo es encontrar defectos que puedan ser perjudiciales para el software.

Error vs Defecto

El **error** se descubre en la misma etapa en la que estoy trabajando.

El **defecto** es un error no detectado que se traslado a una etapa siguiente. **En el testing encontramos defectos.**

Aspectos del testing:

- Severidad: tiene que ver con la gravedad del defecto
 - Bloqueante: no se puede seguir con el caso de prueba hasta que se resuelva
 - Crítico: compromete el resultado del caso de prueba
 - Mayor
 - Menor
 - Cosmético: algo de la presentación del software
- Prioridad: tiene que ver con la urgencia con la que se debe resolver el defecto
 - Urgente
 - Alta
 - Media
 - Baja

Niveles de prueba

1. **Pruebas unitarias:** se prueba el componente individualmente. Se encuentran errores más que defectos, es lo que hace el desarrollador luego de desarrollar el componente

2. **Pruebas de integración:** integrar los componentes ya probados en la prueba unitaria
3. **Pruebas de sistema:** más amplias, se prueba el sistema en toda su escala
4. **Pruebas de aceptación:** son ejecutadas por el usuario final, el que va a usar el software. El cliente verificar que el software cumple con los requisitos funcionales establecidos

Ambientes del desarrollo

1. **Desarrollo:** se escribe el código
2. **Prueba:** se hacen por ejemplo pruebas de integración, no tiene todas las características del entorno de producción
3. **Pre-producción:** lo más parecido al producción, se hacen las pruebas de sistemas
4. **Producción:** entorno final donde esta el software que usa el cliente

Caso de prueba

Un **caso de prueba** es un conjunto de condiciones o acciones predefinidas que se ejecutan para verificar si una funcionalidad del software funciona correctamente.

- Precondiciones
- Condiciones: contexto del sistema que tengo que tener en cuenta para poder hacer la prueba → características de la entrada
- Resultado esperado: lo que efectivamente debe pasar al probar la funcionalidad

La buena definición de los casos de prueba nos **ayuda a reproducir los defectos**.

Se debe lograr definir la menor cantidad de caso de pruebas (menor esfuerzo posible), pero que permitan cubrir el testing del software de la manera más completa posible.

Derivación de Casos de prueba:

1. Desde documentos del cliente
2. Desde información del relevamiento
3. Desde requerimientos
4. Desde especificaciones de la programación
5. Desde el código

Estrategias

Caja Blanca: mira adentro del código, en la estructura interna del software

Caja Negra:

- Basado en especificaciones
 - Partición de equivalencias
 - Análisis de los valores límites (es donde más encontramos los bugs)
- Basados en experiencia
 - Adivinanza de defectos
 - Testing exploratorio

Ciclo de Prueba

Un ciclo es la ejecución de todo un conjunto de casos de prueba establecidos aplicados a una misma versión.

Regresión → cuando corregimos un defecto, normalmente pueden ingresar más, por lo que al corregir se debe realizar una verificación total de la nueva versión

Proceso de pruebas

1. Planificación → plan de prueba
2. Identificación
3. Especificación

4. Ejecución
5. Análisis de falla → es resultado esperado??
6. Fin de las pruebas

Verificación

apunta a evaluar si estamos construyendo el sistema correctamentes

¿El sistema esta siendo construído libre de defectos?

Validación

verificar si estamos construyendo el sistema correcto

¿Es lo que el cliente quiere?

Ciclo de vida del software

objetivo: incluir las actividades de testing lo más antes posible

si conozco los requerimientos / user stories → puedo empezar a escribir casos de prueba

MITOS

- El testing es probar que el software funciona
- El testing arranca cuando el código esta listo
- El tester es el enemigo del programador
- Testing = CALIDAD DE PRODUCTO / PROCESO → asegura que el software sea confiable, pero no asegura la calidad

Principios del Testing

- Un programador debe evitar probar su propio código (a excepción de las pruebas unitarias)

- No planificar el esfuerzo del testing sobre la suposición de que no se van a encontrar defectos

Smoke Test

Corrida rápida por el software, probando en términos generales

Es superficial

Tipos de pruebas

- Testing funcional → relacionada a los requerimientos / procesos de negocio
- Testing no funcional → no tiene que ver con qué hace el sistema (funcional) si no a cómo lo hace
- Performance
- Interfaz de usuario
- Configuración → cómo trabaja el sistema en cierta configuración

TDD: Test Driven Development

Se escribe el código a partir del desarrollo de los Caso de prueba

1- Escribo la prueba

2- Escribo el código para que pase la prueba exitosamente → Refactorización:

CLEAN = code + tests